

Full Stack Developer Technical Assignment

DURATION

24-48 hours

BACKEND

Java + Spring Boot

FRONTEND

Angular

FOCUS

REST, SQL, UI, Tests

Context

Your team supports an internal analytics portal used by business users to monitor operational KPIs across multiple systems. The portal currently relies on manual spreadsheets. Your task is to build a small, production-minded full stack application that stores, filters, and summarizes vehicle feature forecast records.

We are looking for clear design choices, maintainable code, practical testing, and a usable interface rather than a large number of features.

Business Problem

Build a **Vehicle Feature Forecast Dashboard**. Users should be able to view forecast records, filter them by a few business dimensions, inspect summary metrics, and add new records through the application.

Example use case:

A product analyst wants to understand projected adoption of vehicle features by model, region, and quarter without editing spreadsheets directly.

Recommended Tech Stack

Layer	Expected Technology	Notes
Backend	Java 17 or 21, Spring Boot	Use REST APIs, validation, service layer, and repository layer.
Database	PostgreSQL, H2, or another SQL database	H2 is acceptable for speed. PostgreSQL with Docker is a plus.
Frontend	Angular, TypeScript	Angular Material or a similar component library may be used.
Testing	JUnit, Mockito, Angular unit tests	Prioritize meaningful coverage over quantity.

Functional Requirements

1. Forecast Record Management

- Display a paginated or scrollable table of vehicle feature forecast records.
- Create a new forecast record.
- Validate required fields on both frontend and backend.

2. Search and Filtering

- Filter records by `region` and `modelYear`.
- Support a simple free-text search across feature name or vehicle model.
- Allow users to clear filters quickly.

3. Summary Dashboard

- Show total projected volume for the current filtered result set.
- Show average adoption percentage for the current filtered result set.
- Display these metrics as simple summary cards in Angular.

4. Seed Data

- Provide a backend seed dataset loaded at startup.
- Include at least 10 sample records covering multiple regions, vehicle models, quarters, and feature codes.

Suggested Data Model

Field	Type	Required	Description
<code>id</code>	Long / UUID	Yes	Unique identifier.
<code>vehicleModel</code>	String	Yes	Vehicle model name.
<code>modelYear</code>	Integer	Yes	Model year, for example 2026.
<code>region</code>	String	Yes	NA, EU, APAC, LATAM, or another sensible set.
<code>quarter</code>	String	Yes	Forecast quarter, for example 2026-Q1.
<code>featureCode</code>	String	Yes	Short code such as ADAS-PKG or EV-RANGE.
<code>featureName</code>	String	Yes	Human-readable feature name.
<code>projectedVolume</code>	Integer	Yes	Projected number of vehicles.
<code>adoptionRate</code>	Decimal	Yes	Percentage from 0 to 100.
<code>lastUpdatedBy</code>	String	No	User or system that last modified the record.

Backend Requirements

- Expose REST endpoints for listing, filtering, creating records, and summary metrics.
- Use a layered architecture such as controller, service, repository, DTO, and mapper where appropriate.
- Apply backend validation using Bean Validation or an equivalent approach.
- Return useful error responses for validation failures.
- Use SQL persistence through Spring Data JPA, JDBC, or another reasonable Spring-supported approach.
- Include unit tests for one important service method or controller path.

Suggested API Endpoints

Method	Endpoint	Purpose
GET	/api/forecasts	List records with optional filters and search.
POST	/api/forecasts	Create a forecast record.
GET	/api/forecasts/summary	Return dashboard metrics for current filters.

Frontend Requirements

- Create an Angular application with a clear component organization.
- Build one dashboard page with filter controls, summary cards, a data table, and a create form.
- Show basic loading, empty, and error states.
- Use typed services and models for API communication.
- Include at least one meaningful frontend test for a component or service.

Non-Functional Requirements

Code Quality

- Readable naming and small, focused functions.
- Clear separation of frontend and backend responsibilities.
- No hardcoded API URLs scattered throughout components.
- Consistent formatting and linting where available.

Operational Thinking

- Provide clear local setup instructions.
- Include environment configuration where useful.
- Docker Compose is optional but appreciated.
- GitHub Actions workflow is optional but appreciated.

Deliverables

- Source code in a Git repository or compressed archive.
- `README.md` with setup, run, and test instructions.
- Assumptions made where requirements are unclear, documented in `README.md`.
- Brief architecture notes explaining key backend and frontend decisions.
- Sample data or seed script.
- Tests that can be run locally.
- Optional: screenshots or a short screen recording of the working application.

Bonus Items

Bonus items are not required. They can strengthen a submission when the core requirements are already complete.

- Docker Compose setup for backend, frontend, and database.

- GitHub Actions workflow that runs backend and frontend tests.
- Edit existing forecast records.
- CSV import with row-level validation feedback.
- Audit fields such as created date, updated date, and updated by.
- Simple chart or visual summary in Angular.

Submission Instructions

1. Commit code to a repository or provide a compressed archive.
2. Ensure the application can be run from a clean checkout using the README instructions.
3. Assume things where unclear and document those assumptions, shortcuts, or incomplete items in `README.md`.

Expected outcome: A reviewer should be able to run the backend, run the Angular frontend, load sample forecast data, filter records, view summary metrics, create a record, and run the included tests.